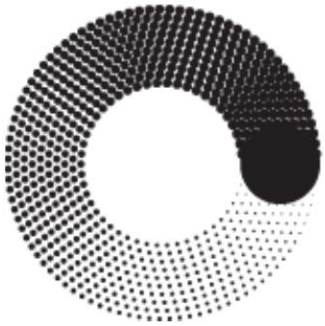
Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования



МОСКОВСКИЙ ПОЛИТЕХ

Факультет информационных технологий

Кафедра «Инфокогнитивные технологии»

Образовательная программа (профиль):
«Разработка и интеграция бизнес-приложений»

ОТЧЕТ

по лабораторной работе 4

на тему: «Создание диаграмм с применением унифицированного языка моделирования (UML)»

по дисциплине: «Моделирование бизнес-процессов»

Выполнил: Андрианова Мария
Студент

Москва
2026

Содержание

1. Определение основных бизнес-процессов и постановка задачи на проектирование……3
2. Диаграмма прецедентов……..…….....3
3. Диаграмма классов…….....7
4. Вывод ……11

1. Определение основных бизнес-процессов и постановка задачи на проектирование

Основные бизнес-процессы школы танцев, реализуемые с использованием ИС:

- Управление данными об учениках
- Учет абонементов
- Управление типами занятий
- Формирование и ведение расписания
- Управление залами
- Ведение занятий преподавателем
- Взаимодействие с учениками (запись, отмена, просмотр)
- Отчетность и аналитика

Постановка задачи на проектирование ИС:

Разработать информационную систему для автоматизации деятельности школы танцев, обеспечивающую:

- централизованное хранение данных об учениках, преподавателях, абонементах, расписании, залах и типах занятий;
- запись учеников на занятия через личный кабинет;
- контроль посещаемости и остатка занятий по абонементу;
- формирование отчетов для руководства;
- разграничение прав доступа для трех типов пользователей: администратор, преподаватель, ученик.

2. Диаграмма прецедентов

Диаграмма прецедентов информационной системы школы танцев представлена на рисунке 1.

Актеры:

- Ученик — клиент школы, посещающий занятия на основании абонемента

- Преподаватель — сотрудник, проводящий занятия
- Администратор — сотрудник, управляющий системой
- Система — технический актёр, выполняющий автоматические операции без участия человека.

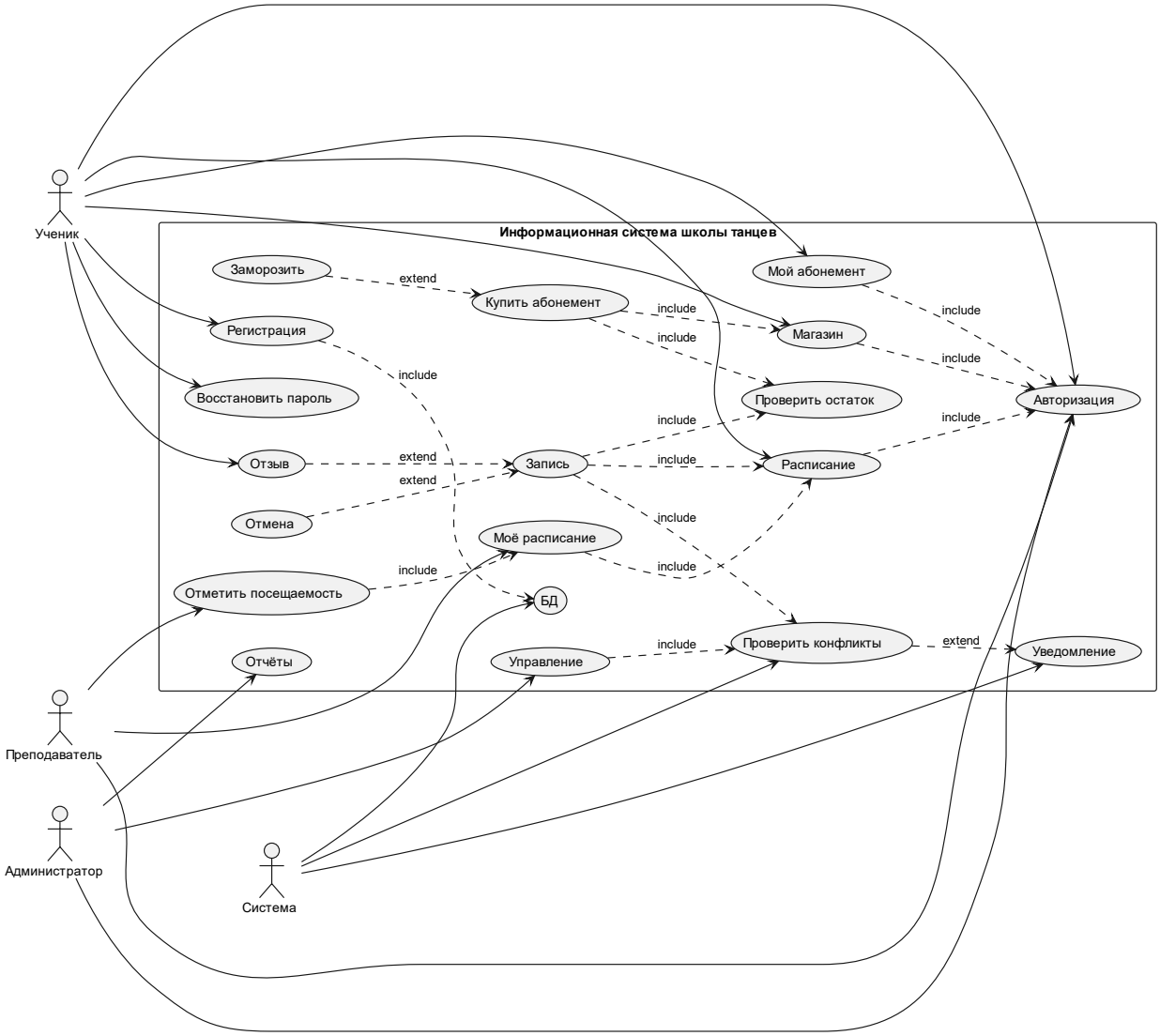


Рисунок 1 – Диаграмма прецедентов

Прецеденты актёра «Ученик»:

Актёр «Ученик» взаимодействует с прецедентами: «Регистрация», «Авторизация», «Восстановить пароль», «Магазин», «Расписание», «Мой абонемент», «Отзыв». Через прецедент «Магазин» ученик переходит к покупке абонемента, а через прецедент «Расписание» — к записи на занятие.

Прецеденты актёра «Преподаватель»:

Актёр «Преподаватель» взаимодействует с прецедентами: «Авторизация», «Моё расписание», «Отметить посещаемость». Прецедент «Моё расписание» позволяет преподавателю просматривать только свои занятия (в отличие от общего расписания). Прецедент «Отметить посещаемость» используется для фиксации присутствия учеников.

Прецеденты актёра «Администратор»:

Актёр «Администратор» взаимодействует с прецедентами: «Авторизация», «Управление», «Отчёты». Прецедент «Управление» объединяет функции по управлению абонементами, расписанием, залами и пользователями. Прецедент «Отчёты» позволяет формировать аналитическую информацию о деятельности школы.

Прецеденты актёра «Система»:

Актёр «Система» взаимодействует с прецедентами: «БД», «Уведомление», «Проверить конфликты». Эти прецеденты выполняются автоматически: запись и чтение данных из базы данных, отправка уведомлений пользователям, проверка конфликтов расписания (например, пересечение занятий в одном зале).

Связи между прецедентами:

Прецедент «Купить абонемент» включается в прецедент «Магазин» (связь include), так как покупка является действием внутри магазина. Прецедент «Проверить остаток» включается в прецеденты «Купить абонемент» и «Запись», поскольку перед этими операциями система должна проверить наличие действующего абонемента с положительным остатком занятий.

Прецедент «Заморозить» расширяет прецедент «Купить абонемент» (связь extend), так как заморозка возможна только после приобретения абонемента. Прецедент «Отмена» расширяет прецедент «Запись», поскольку отменить можно только существующую запись. Прецедент «Отзыв» также расширяет прецедент «Запись», так как оставить отзыв можно только после посещения занятия.

Прецедент «Моё расписание» включает прецедент «Расписание», так как для отображения персонального расписания преподавателя необходимо обратиться к общему расписанию занятий. Прецедент «Отметить посещаемость» включает прецедент «Моё расписание», поскольку отметка возможна только для занятия, которое есть в расписании преподавателя.

Прецедент «Управление» включает прецедент «Проверить конфликты», так как при изменении расписания система автоматически проверяет возможные пересечения. Прецедент «Проверить конфликты» расширяет прецедент «Уведомление» (связь extend), так как при обнаружении конфликта система отправляет уведомление администратору.

Прецедент «Регистрация» включает прецедент «БД», поскольку данные нового пользователя сохраняются в базу данных. Прецедент «Авторизация» включается в прецеденты «Магазин» и «Расписание», так как эти действия требуют предварительной аутентификации пользователя. Прецедент «Мой абонемент» также включает прецедент «Авторизация».

Границы системы:

Все перечисленные прецеденты находятся внутри границы системы — прямоугольника с именем «Информационная система школы танцев». Актёры расположены за границами системы, так как являются внешними сущностями, взаимодействующими с системой, но не принадлежащими ей.

3. Диаграмма классов

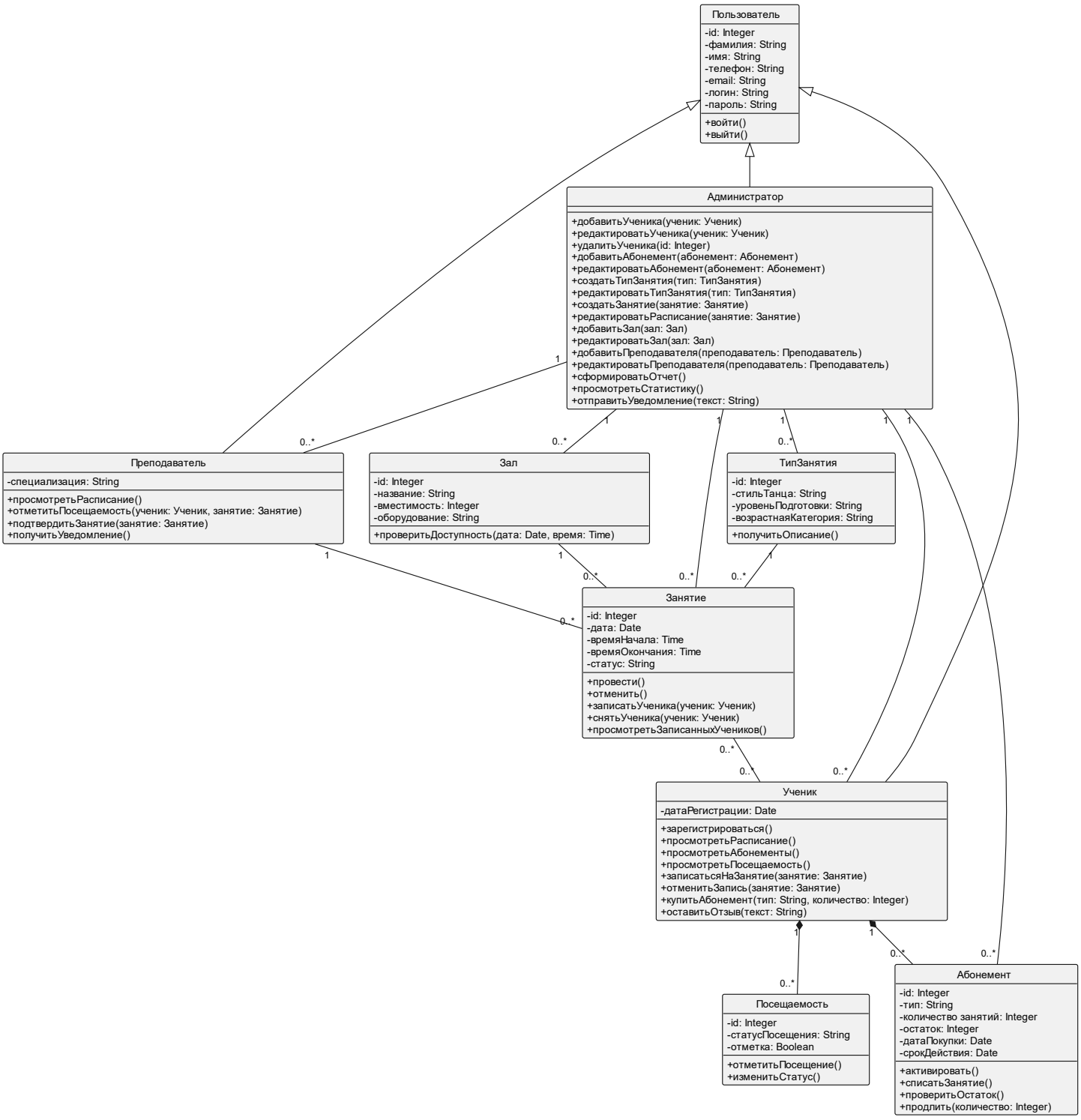


Рисунок 2 – Диаграмма классов

Разработанная на рисунке 2 диаграмма классов содержит 9 классов, их атрибуты, операции и отношения между ними.

Класс «Пользователь» является базовым для всех типов пользователей системы. Он содержит общие атрибуты: идентификатор, фамилию, имя, телефон, email, логин и пароль. Операции: войти в систему и выйти из системы.

Класс «Ученик» наследует класс «Пользователь». Дополнительный атрибут — дата регистрации. Операции: зарегистрироваться, просмотреть расписание, просмотреть свои абонементы, просмотреть историю посещаемости, записаться на занятие, отменить запись, купить абонемент, оставить отзыв.

Класс «Преподаватель» наследует класс «Пользователь». Дополнительный атрибут — специализация. Операции: просмотреть своё расписание, отметить посещаемость учеников на занятии, подтвердить проведение занятия, получить уведомление.

Класс «Администратор» наследует класс «Пользователь». Операции: добавление, редактирование и удаление учеников; добавление, редактирование и удаление абонементов; создание и редактирование типов занятий; создание занятий и редактирование расписания; добавление и редактирование залов; добавление и редактирование преподавателей; формирование отчетов, просмотр статистики, отправка уведомлений.

Класс «Абонемент» содержит атрибуты: идентификатор, тип абонемента, количество занятий по абонементу, остаток занятий, дата покупки, срок действия. Операции: активировать абонемент, списать одно занятие, проверить остаток занятий, продлить абонемент.

Класс «Занятие» содержит атрибуты: идентификатор, дата проведения, время начала, время окончания, статус занятия (запланировано, проведено, отменено). Операции: провести занятие, отменить занятие, записать ученика на занятие, снять ученика с занятия, просмотреть список записанных учеников.

Класс «ТипЗанятия» содержит справочные атрибуты: идентификатор, стиль танца (хип-хоп, бальные и т.д.), уровень подготовки (новичок, средний,

продвинутый), возрастная категория (дети, взрослые). Операция: получить описание типа занятия.

Класс «Зал» содержит атрибуты: идентификатор, название зала, вместимость (максимальное количество человек), оборудование. Операция: проверить доступность зала на конкретные дату и время.

Класс «Посещаемость» содержит атрибуты: идентификатор, статус посещения (пришёл, опоздал, не пришёл), отметка (посетил или нет). Операции: отметить посещение, изменить статус посещения.

Отношения между классами

Класс «Пользователь» находится в отношении наследования с классами «Ученик», «Преподаватель» и «Администратор». Это означает, что каждый из них является частным случаем пользователя и наследует все его атрибуты и функции.

Класс «Ученик» связан с классом «Абонемент» отношением композиции. Один ученик может иметь несколько абонементов (кратность 0..*), а каждый абонемент принадлежит только одному ученику (кратность 1). Композиция означает, что абонемент не может существовать без ученика — при удалении ученика его абонементы также удаляются.

Класс «Ученик» также связан с классом «Посещаемость» отношением композиции. Один ученик может иметь несколько отметок посещаемости (кратность 0..*), а каждая отметка относится только к одному ученику (кратность 1). Отметка не может существовать без ученика.

Класс «Зал» связан с классом «Занятие» ассоциацией. Один зал может использоваться для проведения нескольких занятий (кратность 0..*), а каждое занятие проводится только в одном зале (кратность 1). Это простая связь без владения — зал существует независимо от занятий.

Класс «Преподаватель» связан с классом «Занятие» ассоциацией. Один преподаватель может вести несколько занятий (кратность 0..*), а каждое занятие ведёт только один преподаватель (кратность 1). Это связь «ведёт», преподаватель не владеет занятиями.

Класс «ТипЗанятия» связан с классом «Занятие» ассоциацией. Один тип занятия может соответствовать нескольким занятиям (кратность 0..*), а каждое занятие имеет только один тип (кратность 1). Тип занятия — это справочная информация, которая может существовать без занятий.

Класс «Занятие» связан с классом «Ученик» ассоциацией «многие ко многим». Одно занятие могут посещать несколько учеников (кратность 0..*), и один ученик может записаться на несколько занятий (кратность 0..*). Это связь через запись, ни один из классов не владеет другим.

Класс «Администратор» связан ассоциациями со всеми управляемыми классами: «Ученик», «Преподаватель», «Абонемент», «ТипЗанятия», «Занятие», «Зал». Во всех случаях кратность со стороны администратора — 1 (один администратор управляет), а со стороны управляемого класса — 0..* (может быть много объектов).

4. Вывод

В ходе выполнения лабораторной работы была спроектирована информационная система для автоматизации деятельности школы танцев с использованием унифицированного языка моделирования UML.

На первом этапе были выделены основные бизнес-процессы школы: управление учениками, учёт абонементов, управление типами занятий, формирование расписания, управление залами, проведение занятий и отчётность. Сформулирована постановка задачи на проектирование информационной системы, определены функциональные требования и три типа пользователей: ученик, преподаватель и администратор.

На втором этапе разработана диаграмма прецедентов, которая включает трёх актёров и 18 прецедентов. Диаграмма отражает все ключевые сценарии взаимодействия пользователей с системой: от регистрации ученика до формирования отчётов администратором. Отношения включения и расширения показывают логические связи между прецедентами.

На третьем этапе построена диаграмма классов, содержащая 9 классов, их атрибуты, операции и отношения между ними. Классы «Ученик», «Преподаватель» и «Администратор» наследуют общие атрибуты и методы от базового класса «Пользователь», что исключает дублирование и соответствует принципам объектно-ориентированного проектирования. Отношения композиции применены для связей «Ученик — Абонемент» и «Ученик — Посещаемость», так как эти объекты не могут существовать друг без друга. Остальные связи реализованы как ассоциации, что соответствует рекомендациям UML по избеганию неоднозначной агрегации.

Важность проведенной работы для деятельности компании

Разработанные UML-диаграммы имеют практическую значимость для школы танцев по следующим причинам:

1. Наглядность и стандартизация. Диаграммы прецедентов и классов дают полное и наглядное представление о том, какие функции должна

выполнять информационная система и из каких сущностей она состоит. Это единый язык общения между заказчиком, аналитиками и разработчиками.

2. Основа для реализации. Диаграмма классов является готовой основой для проектирования базы данных и написания программного кода. Каждый класс на диаграмме становится отдельным модулем, атрибуты — полями таблиц, операции — методами.
3. Выявление пробелов. В процессе моделирования были выявлены недостающие функции (например, просмотр расписания у ученика и преподавателя, отправка уведомлений), которые были добавлены до начала разработки. Это позволяет избежать доработок на поздних стадиях.
4. Снижение рисков. Чёткое разграничение прав доступа между тремя типами пользователей, заложенное в диаграмме классов, обеспечивает безопасность данных и исключает несанкционированные действия.
5. Масштабируемость. Использование наследования от базового класса «Пользователь» позволяет легко добавлять новые роли (например, «Бухгалтер» или «Методист») без перестройки всей модели.

Таким образом, разработанные диаграммы UML являются не просто учебным заданием, а полноценным проектом информационной системы, который может быть передан в разработку. Их внедрение позволит школе танцев автоматизировать рутинные операции, повысить удовлетворённость клиентов и улучшить финансовые показатели за счёт контроля рентабельности занятий и снижения числа отмен.